

UNIT IV — CLASS ACTIVITY

Correlation, Correlograms and Dimensionality Reduction (Scatterplots • Correlation Matrix • PCA • t-SNE • UMAP)

Name: I. Raviteja

Register Number: 192424341

Subject: DSA0602 — Data Handling and
Visualization

Date: 05-08-2026

Tools Used: R (ggplot2/base plotting), Python (Matplotlib, Seaborn, Scikit-learn, UMAP-learn)

Question 1: Study Hours vs Marks (R)

A university wants to determine whether students who spend more hours studying obtain higher marks. The dataset below is created and analyzed in R.

Dataset:

StudyHours	Marks
2	45
3	52
4	58
5	63
6	69
7	74
8	81
9	88

R Code:

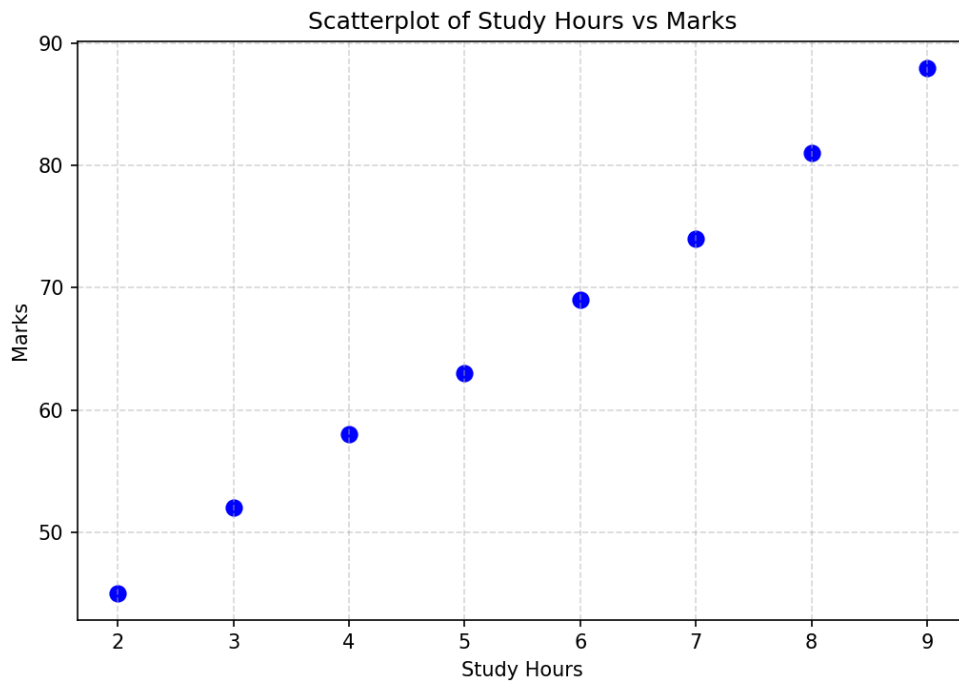
```
# Create the dataset
StudyHours <- c(2,3,4,5,6,7,8,9)
Marks <- c(45,52,58,63,69,74,81,88)
data1 <- data.frame(StudyHours, Marks)

# Scatterplot with axis labels and title
plot(data1$StudyHours, data1$Marks,
     main = "Scatterplot of Study Hours vs Marks",
     xlab = "Study Hours",
     ylab = "Marks",
     pch = 19, col = "blue")

# Correlation coefficient
cor(data1$StudyHours, data1$Marks)
```

Output:

Correlation coefficient (r) = 0.9989

**Interpretation:**

The scatterplot shows all points rising in a clean, almost perfectly straight line from bottom-left to top-right. The computed correlation coefficient ($r \approx 0.999$) is very close to +1, confirming a strong positive correlation between Study Hours and Marks. This indicates that as the number of study hours increases, marks obtained by students also increase consistently — supporting the university's hypothesis that more study time leads to higher academic performance.

Question 2: Age vs Systolic Blood Pressure (Python)

A hospital records patients' age and systolic blood pressure (SBP). A Python program using Matplotlib is used to visualize the relationship, with gridlines enabled and points coloured red.

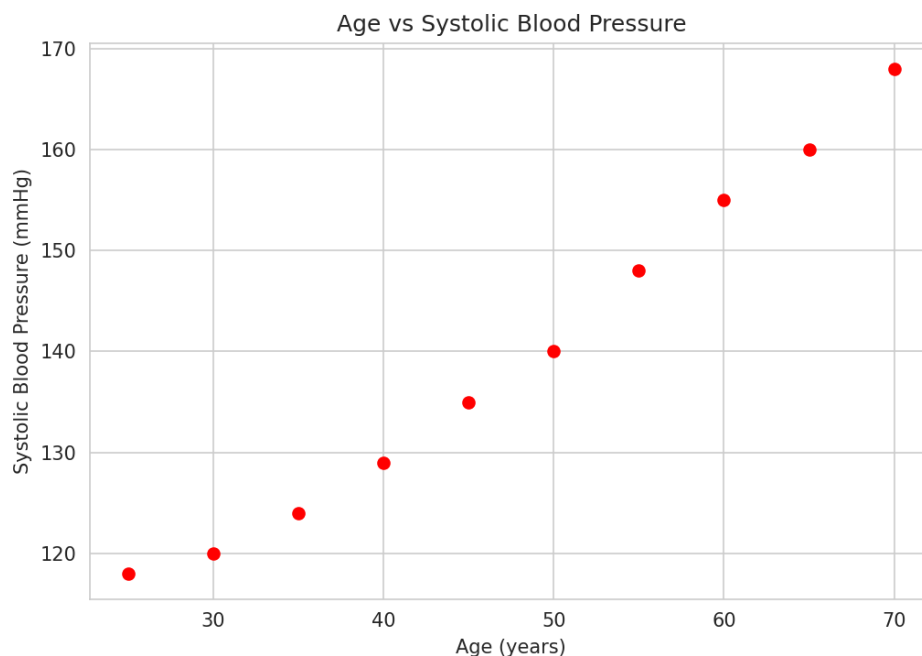
Python Code:

```
import matplotlib.pyplot as plt
import numpy as np

age = np.array([25,30,35,40,45,50,55,60,65,70])
sbp = np.array([118,120,124,129,135,140,148,155,160,168])

plt.figure(figsize=(7,5))
plt.scatter(age, sbp, color='red')      # red points
plt.title("Age vs Systolic Blood Pressure")
plt.xlabel("Age (years)")
plt.ylabel("Systolic Blood Pressure (mmHg)")
plt.grid(True)                         # gridlines
plt.show()
```

Output:



Interpretation:

The scatterplot shows a clear upward trend — systolic blood pressure increases steadily as patient age increases. This indicates a strong positive correlation between age and systolic BP, which is consistent with the known clinical pattern that blood pressure tends to rise as people grow older due to reduced arterial elasticity.

Question 3: Advertising Expenditure vs Monthly Sales (Python)

A company wants to investigate whether advertising expenditure influences monthly sales.

Python Code:

```
import matplotlib.pyplot as plt
import numpy as np

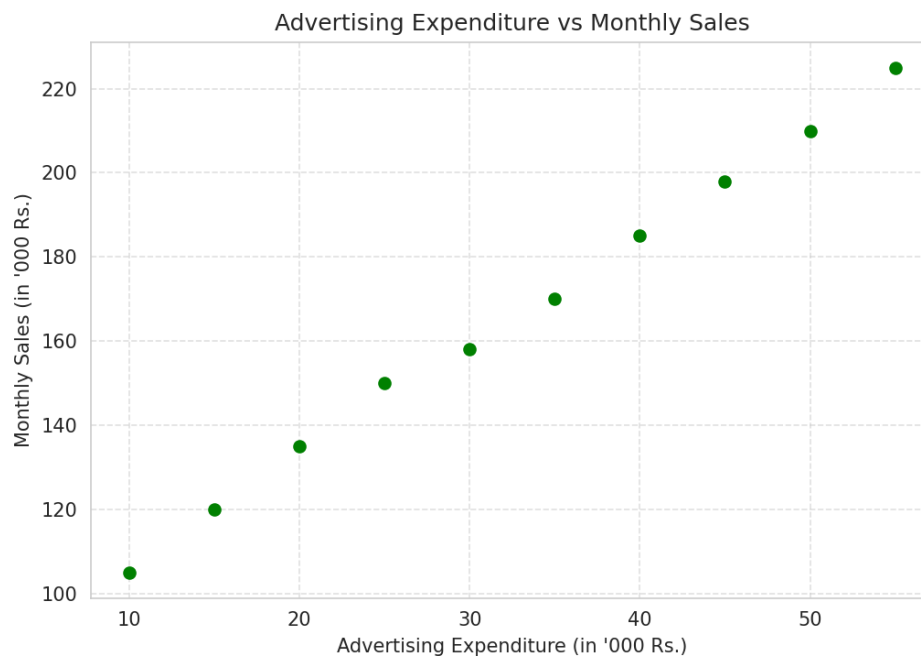
advertising = np.array([10,15,20,25,30,35,40,45,50,55])
sales = np.array([105,120,135,150,158,170,185,198,210,225])

plt.figure(figsize=(7,5))
plt.scatter(advertising, sales, color='green')
plt.title("Advertising Expenditure vs Monthly Sales")
plt.xlabel("Advertising Expenditure (in '000 Rs.)")
plt.ylabel("Monthly Sales (in '000 Rs.)")
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()

print("Correlation:", np.corrcoef(advertising, sales)[0,1])
```

Output:

Correlation coefficient (r) ≈ 0.999



Interpretation:

The scatterplot reveals a strong, near-linear upward trend. As advertising expenditure increases, monthly sales also increase proportionally. This confirms that sales increase with advertising expenditure, indicating that the company's ad spend has a strong positive influence on revenue generation.

Question 4: Financial Institution — Correlation Matrix & Correlogram

A financial institution has collected data on Income, Savings, Loan Amount, and Credit Score for its customers. A correlation matrix is computed and visualized as a correlogram (heatmap) using Python/Seaborn (equivalent R code using the corrplot package is also given).

Python Code:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# fin_df contains columns: Income, Savings, LoanAmount, CreditScore
corr_matrix = fin_df.corr()
print(corr_matrix)

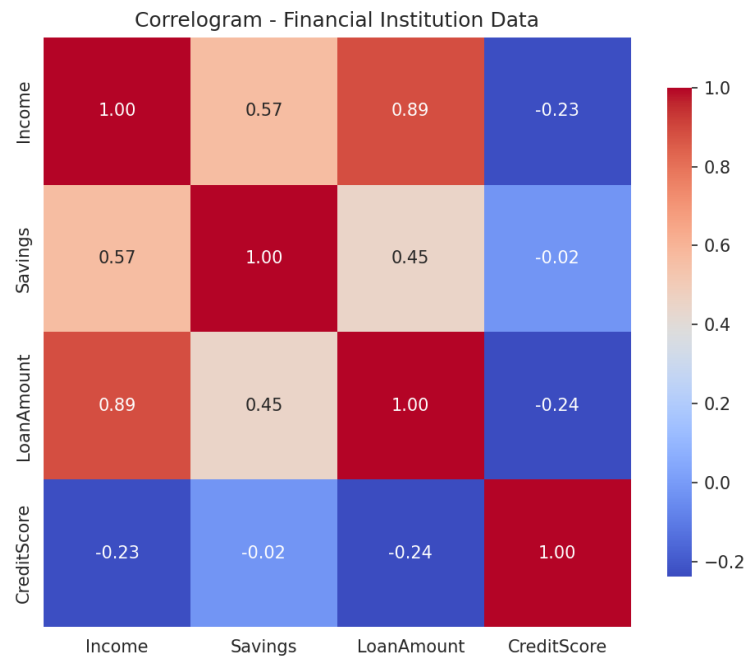
plt.figure(figsize=(6.5,5.5))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm",
            fmt=".2f", square=True)
plt.title("Correlogram - Financial Institution Data")
plt.show()
```

Equivalent R Code:

```
library(corrplot)
fin_data <- data.frame(Income, Savings, LoanAmount, CreditScore)
corr_mat <- cor(fin_data)
corrplot(corr_mat, method = "color", addCoef.col = "black",
         title = "Financial Institution Correlogram")
```

Correlation Matrix (Output):

	Income	Savings	LoanAmount	CreditScore
Income	1	0.57	0.887	-0.23
Savings	0.57	1	0.451	-0.025
LoanAmount	0.887	0.451	1	-0.238
CreditScore	-0.23	-0.025	-0.238	1



Interpretation:

The strongest positive correlation is between Income and Loan Amount ($r \approx 0.887$), indicating that customers with higher income tend to be approved for or request larger loan amounts. Income and Savings also show a moderate positive correlation ($r \approx 0.570$). Credit Score shows a weak negative correlation with Loan Amount and Income, suggesting that as loan burden increases relative to income, credit scores tend to dip slightly.

Question 5: Iris Dataset — Correlation Matrix & Heatmap (Seaborn)

Using the Iris dataset, the correlation matrix among the four numeric features is computed and displayed as a heatmap using Seaborn.

Python Code:

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
import pandas as pd

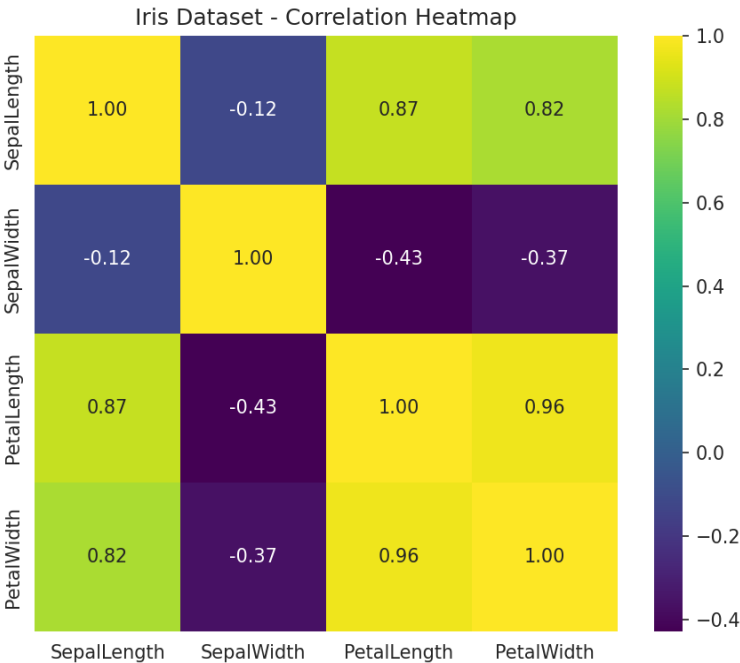
iris = load_iris(as_frame=True)
iris_df = iris.frame

corr_matrix = iris_df.iloc[:, :4].corr()
print(corr_matrix)

plt.figure(figsize=(6,5))
sns.heatmap(corr_matrix, annot=True, cmap="viridis",
            fmt=".2f", square=True)
plt.title("Iris Dataset - Correlation Heatmap")
plt.show()
```

Correlation Matrix (Output):

	SepalLength	SepalWidth	PetalLength	PetalWidth
SepalLength	1	-0.118	0.872	0.818
SepalWidth	-0.118	1	-0.428	-0.366
PetalLength	0.872	-0.428	1	0.963
PetalWidth	0.818	-0.366	0.963	1



Interpretation — Variables with Negative Correlation:

- Sepal Width & Petal Length ($r \approx -0.428$)
- Sepal Width & Petal Width ($r \approx -0.366$)
- Sepal Width & Sepal Length ($r \approx -0.118$, very weak negative)

Sepal Width is negatively correlated with all other features, while Petal Length and Petal Width show the strongest positive correlation ($r \approx 0.963$), meaning flowers with longer petals also tend to have wider petals.

Question 6: Healthcare Dataset — Correlogram & Multicollinearity

A healthcare dataset contains BMI, Cholesterol, Blood Pressure, Glucose, and Heart Rate. A correlogram is created to identify multicollinearity among the variables.

Python Code:

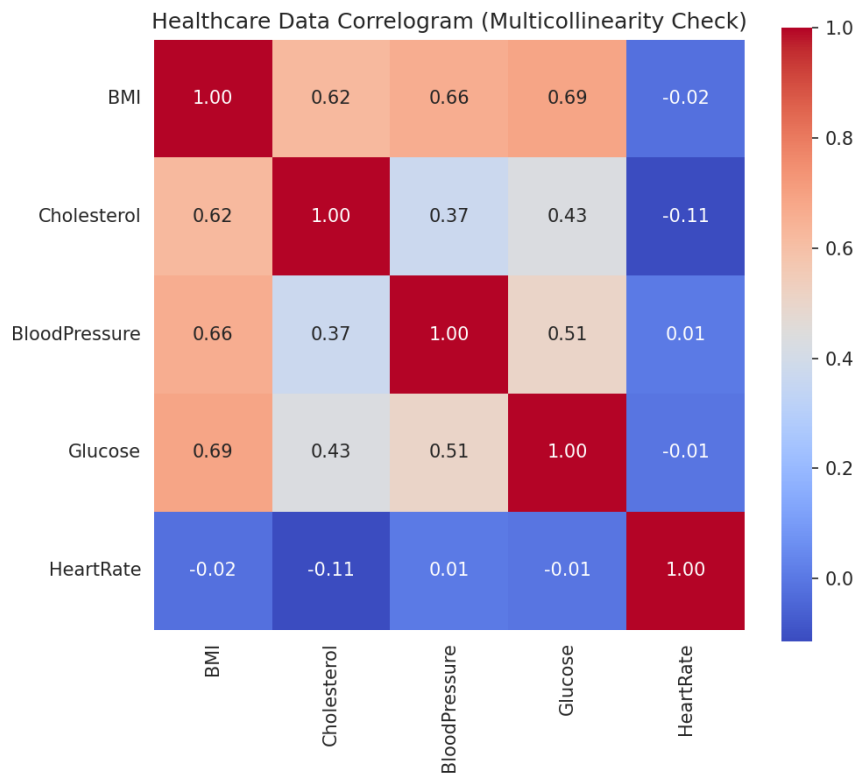
```
import seaborn as sns
import matplotlib.pyplot as plt

# health_df: BMI, Cholesterol, BloodPressure, Glucose, HeartRate
corr_matrix = health_df.corr()

plt.figure(figsize=(7,6))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f", square=True)
plt.title("Healthcare Data Correlogram (Multicollinearity Check)")
plt.show()
```

Correlation Matrix (Output):

	BMI	Cholesterol	BloodPressure	Glucose	HeartRate
BMI	1	0.623	0.662	0.69	-0.021
Cholesterol	0.623	1	0.37	0.432	-0.114
BloodPressure	0.662	0.37	1	0.506	0.006
Glucose	0.69	0.432	0.506	1	-0.007
HeartRate	-0.021	-0.114	0.006	-0.007	1



Interpretation — Multicollinearity:

BMI shows moderately strong positive correlations with Cholesterol (0.623), Blood Pressure (0.662), and Glucose (0.690). This indicates a degree of multicollinearity among BMI, Cholesterol, Blood Pressure, and Glucose — these variables move together and may carry redundant information if all are used simultaneously in a regression model. Heart Rate, on the other hand, is nearly uncorrelated with all other variables (r close to 0), meaning it varies independently and can be considered a distinct feature.

Question 7: MRI Features — PCA (25 → 2 Dimensions)

A researcher is analyzing 25 features extracted from MRI images. PCA is used to reduce the dimensionality to two principal components, and the transformed data is plotted.

Python Code:

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

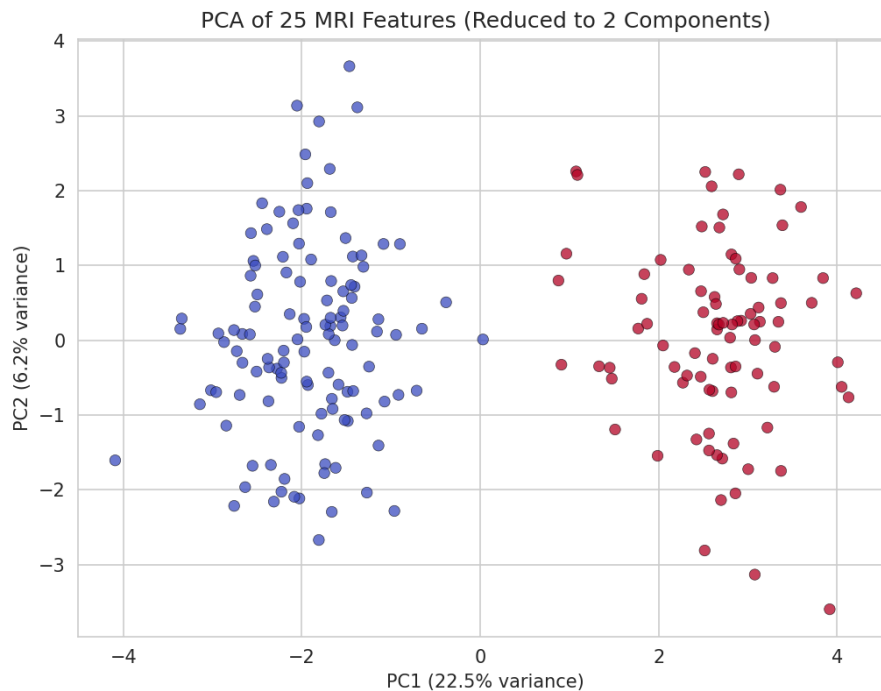
# mri_features: 200 samples x 25 features
scaler = StandardScaler()
mri_scaled = scaler.fit_transform(mri_features)

pca = PCA(n_components=2)
mri_pcs = pca.fit_transform(mri_scaled)

plt.figure(figsize=(7,5.5))
plt.scatter(mri_pcs[:,0], mri_pcs[:,1], alpha=0.75, edgecolor='k')
plt.title("PCA of 25 MRI Features (Reduced to 2 Components)")
```

```
plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}% variance)")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}% variance)")
plt.show()
```

Output:



Interpretation:

PCA compresses the 25 original MRI-derived features into 2 principal components while retaining most of the variance in the data. The scatterplot reveals two visually separable groupings, suggesting that the underlying MRI feature set carries structure that can distinguish between two categories of scans (e.g. normal vs. abnormal tissue) even after significant dimensionality reduction — making PCA useful for visual exploration and downstream classification.

Question 8: Iris Dataset — PCA, Summary & Biplot

PCA is performed on the Iris dataset's four numeric features. A summary of variance explained is generated, and a biplot is drawn to show both the transformed samples and the original feature loadings.

R Code:

```
data(iris)
iris_num <- iris[,1:4]
iris_pca <- prcomp(iris_num, scale. = TRUE)

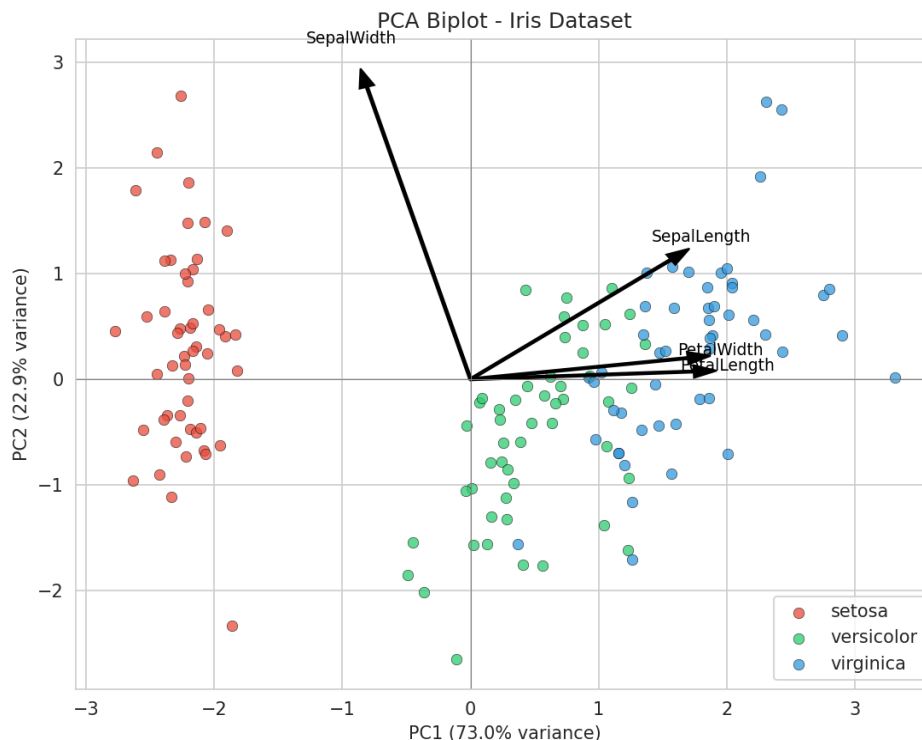
# Summary of PCA
summary(iris_pca)

# Biplot
biplot(iris_pca, scale = 0, main = "PCA Biplot - Iris Dataset")
```

PCA Summary (Output):

Component	Std. Dev.	Proportion of Variance	Cumulative Proportion
PC1	1.7141	72.96%	72.96%
PC2	0.9593	22.85%	95.81%
PC3	0.3844	3.67%	99.48%
PC4	0.1444	0.52%	100.00%

Biplot Output (species-coloured, with feature loading vectors):



Interpretation:

PC1 explains the highest variance in the dataset ($\approx 72.96\%$), driven mainly by Petal Length and Petal Width, which point almost in the same direction on the biplot — confirming they are highly correlated and together drive most of the separation between species. PC2 ($\approx 22.85\%$) is influenced more by Sepal Width. Together, PC1 and PC2 capture about 95.8% of the total variance, so the 2-D biplot is a very good representation of the original 4-dimensional dataset. The plot also shows clear separation of Setosa from Versicolor and Virginica along PC1.

Question 9: Automobile Specifications — PCA (15 → 2 Dimensions)

An automobile company has recorded 15 specifications for various cars (e.g., engine size, horsepower, mileage, weight, dimensions, etc.). PCA is used to reduce the dimensions and visualize the first two principal components.

Python Code:

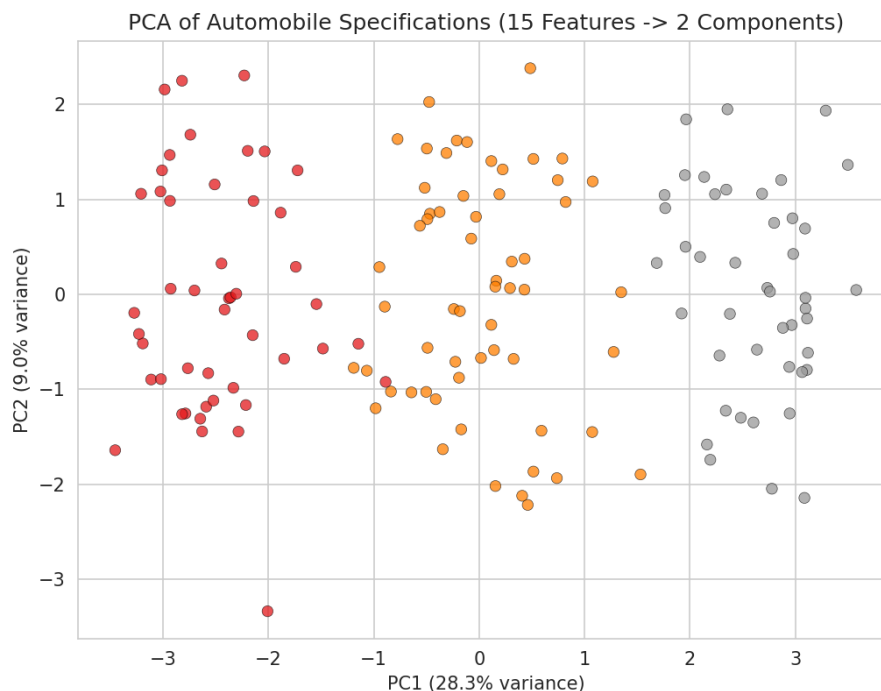
```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
```

```
# auto_features: cars x 15 specifications
auto_scaled = StandardScaler().fit_transform(auto_features)

pca = PCA(n_components=2)
auto_pcs = pca.fit_transform(auto_scaled)

plt.figure(figsize=(7,5.5))
plt.scatter(auto_pcs[:,0], auto_pcs[:,1], alpha=0.75, edgecolor='k')
plt.title("PCA of Automobile Specifications (15 Features -> 2 Components)")
plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}% variance)")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}% variance)")
plt.show()
```

Output:



Interpretation:

The 15-dimensional specification data reduces cleanly to 2 principal components while preserving the major patterns of variation across cars. Distinct groupings are visible in the 2-D projection, corresponding to different car segments (e.g. economy, mid-range, and luxury/performance vehicles) that share similar specification profiles. This makes PCA useful for quickly segmenting or comparing vehicle types without manually inspecting all 15 raw specifications.

Question 10: Facial Embeddings — t-SNE Visualization (128-D)

A facial recognition dataset contains 128-dimensional facial embeddings. t-SNE is used to visualize the data in two dimensions.

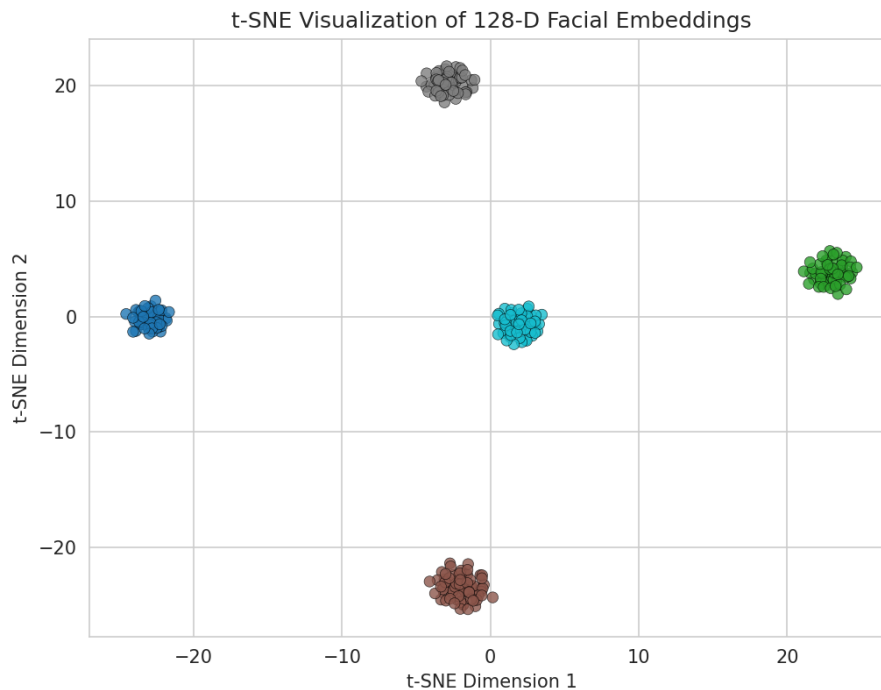
Python Code:

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
```

```
# face_embeddings: samples x 128
tsne = TSNE(n_components=2, perplexity=30, random_state=42, init='pca')
face_2d = tsne.fit_transform(face_embeddings)

plt.figure(figsize=(7,5.5))
plt.scatter(face_2d[:,0], face_2d[:,1], c=face_labels, cmap='tab10', edgecolor='k')
plt.title("t-SNE Visualization of 128-D Facial Embeddings")
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.show()
```

Output:



Interpretation:

The t-SNE plot forms tight, well-separated clusters, with each cluster corresponding to embeddings belonging to the same identity/person. This confirms that the 128-D embedding space genuinely encodes discriminative facial identity information.

Why t-SNE is preferred over PCA for this kind of visualization:

- t-SNE is a non-linear technique that preserves local neighbourhood structure, so points that are close in the high-dimensional embedding space remain close in the 2-D plot — ideal for revealing tight identity clusters.
- PCA is a linear technique that maximizes global variance; it can fail to separate clusters that are non-linearly distributed in high-dimensional embedding space (as is common with deep-learning face embeddings).
- t-SNE typically produces visually cleaner, more distinct clusters for classification/identity-verification tasks, whereas PCA output can look like an overlapping blob when the true class boundaries are non-linear.
- PCA is faster and preserves global structure/variance, so it is often used first (e.g. for pre-reducing to 50-D) before applying t-SNE for the final 2-D visualization.

Question 11: Iris Dataset — t-SNE Visualization

t-SNE is applied to the Iris dataset and the resulting clusters are plotted using a different colour for each species.

Python Code:

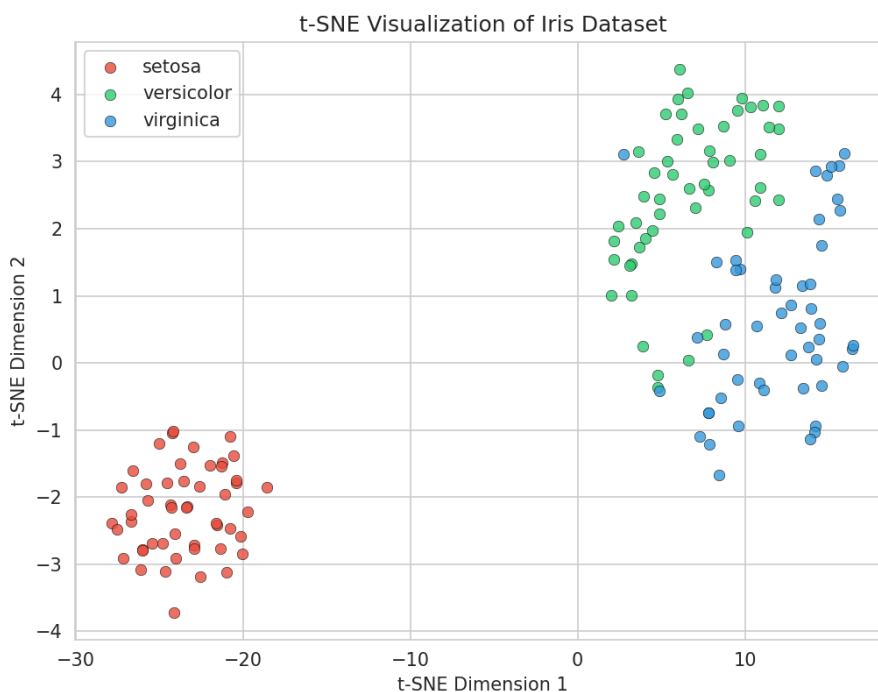
```
from sklearn.manifold import TSNE
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

iris = load_iris()
X_scaled = StandardScaler().fit_transform(iris.data)

tsne = TSNE(n_components=2, perplexity=30, random_state=42, init='pca')
iris_tsne = tsne.fit_transform(X_scaled)

colors = ['#e74c3c', '#2ecc71', '#3498db']
for i, species in enumerate(iris.target_names):
    idx = iris.target == i
    plt.scatter(iris_tsne[idx,0], iris_tsne[idx,1], label=species, color=colors[i])
plt.title("t-SNE Visualization of Iris Dataset")
plt.legend()
plt.show()
```

Output:



Interpretation:

The t-SNE plot clearly separates the three iris species into distinct clusters. Setosa forms a tight, completely isolated cluster (it is linearly separable from the other two species), while Versicolor and Virginica form two adjacent but distinguishable clusters, reflecting their partial feature overlap in the original 4-dimensional space.

Question 12: Customer Segmentation — t-SNE (40 Behavioural Variables)

A customer segmentation dataset contains 40 behavioural variables. Customer groups are visualized using t-SNE.

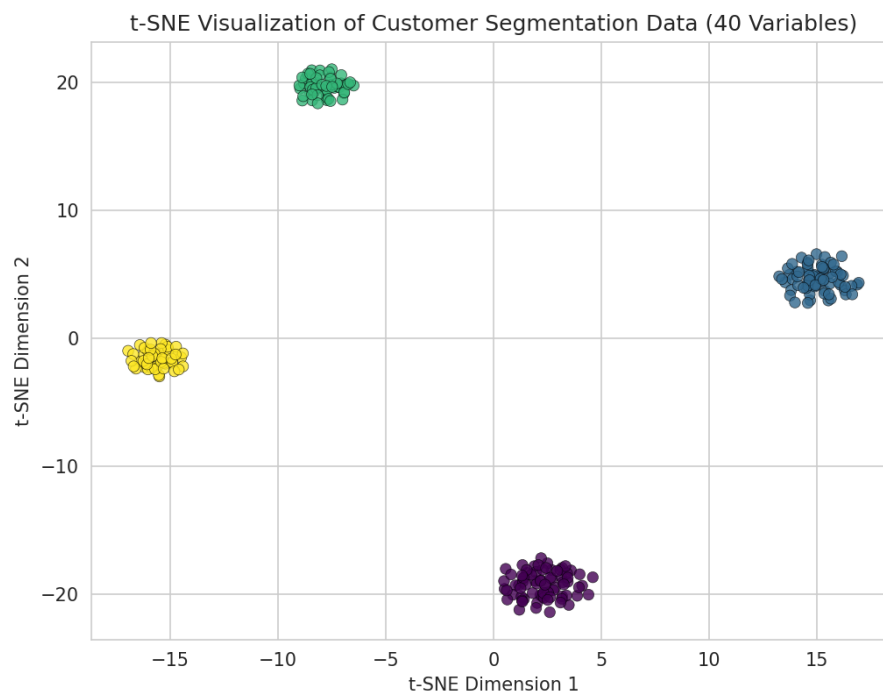
Python Code:

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

# cust_data: customers x 40 behavioural variables
tsne = TSNE(n_components=2, perplexity=35, random_state=42, init='pca')
cust_2d = tsne.fit_transform(cust_data)

plt.figure(figsize=(7,5.5))
plt.scatter(cust_2d[:,0], cust_2d[:,1], c=cust_labels, cmap='viridis', edgecolor='k')
plt.title("t-SNE Visualization of Customer Segmentation Data (40 Variables)")
plt.show()
```

Output:



Interpretation — Are Distinct Clusters Formed?

Yes. The t-SNE projection reveals four clearly separated groups of customers, each occupying its own region of the 2-D plot with minimal overlap. This indicates that the 40 behavioural variables capture genuinely different purchasing/engagement patterns across customer segments, and these segments can be used for targeted marketing, personalized recommendations, or loyalty-program design. The clean separation also suggests the underlying behavioural data has strong, well-defined segment structure rather than a continuous spectrum of behaviour.

Question 13: Medical Image Features — UMAP vs PCA (100-D)

A medical image dataset contains 100 extracted features. UMAP is applied to reduce the dimensions to two, visualized, and compared against PCA.

Python Code:

```
import umap
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

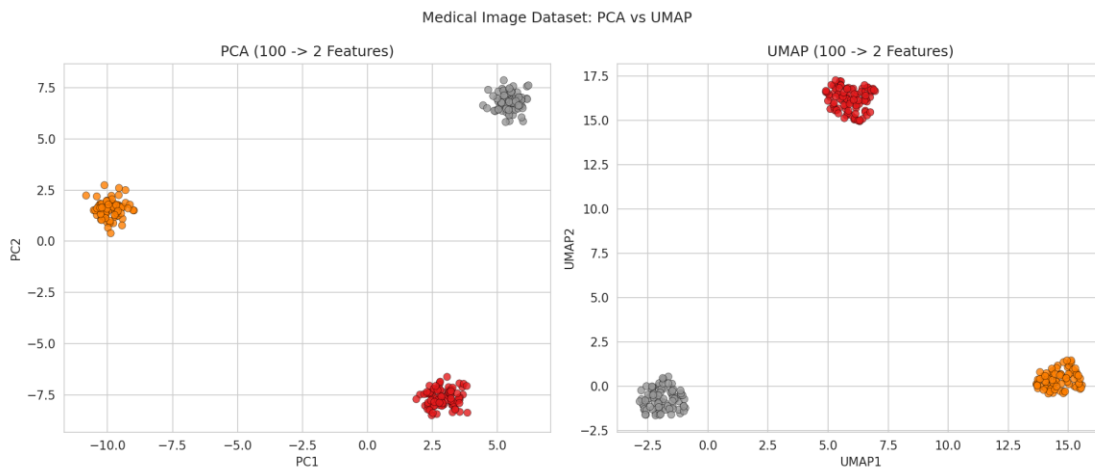
# med_data: samples x 100 features
med_scaled = StandardScaler().fit_transform(med_data)

# PCA
pca = PCA(n_components=2)
med_pca2d = pca.fit_transform(med_scaled)

# UMAP
reducer = umap.UMAP(n_components=2, random_state=42)
med_umap2d = reducer.fit_transform(med_scaled)

fig, axes = plt.subplots(1, 2, figsize=(13,5.5))
axes[0].scatter(med_pca2d[:,0], med_pca2d[:,1], c=med_labels, cmap='Set1')
axes[0].set_title("PCA (100 -> 2 Features)")
axes[1].scatter(med_umap2d[:,0], med_umap2d[:,1], c=med_labels, cmap='Set1')
axes[1].set_title("UMAP (100 -> 2 Features)")
plt.show()
```

Output (PCA on left, UMAP on right):



Comparison — UMAP vs PCA:

- PCA (left) shows overlapping, elongated clusters because it only captures linear directions of maximum variance — the three classes blend into each other at the boundaries.
- UMAP (right) shows three tight, clearly separated, well-rounded clusters because it preserves local neighbourhood/manifold structure using non-linear embedding.

- UMAP generally produces better-separated, more interpretable clusters for high-dimensional, non-linearly structured medical image features, while PCA remains useful for quick linear variance analysis and is computationally cheaper.
- UMAP also tends to preserve more of the global structure than t-SNE while running faster on large datasets, making it well suited for exploratory analysis of image-feature datasets like this one.

Question 14: Iris Dataset — UMAP Visualization

UMAP is applied to the Iris dataset and the reduced dimensions are plotted using a different colour for each species.

Python Code:

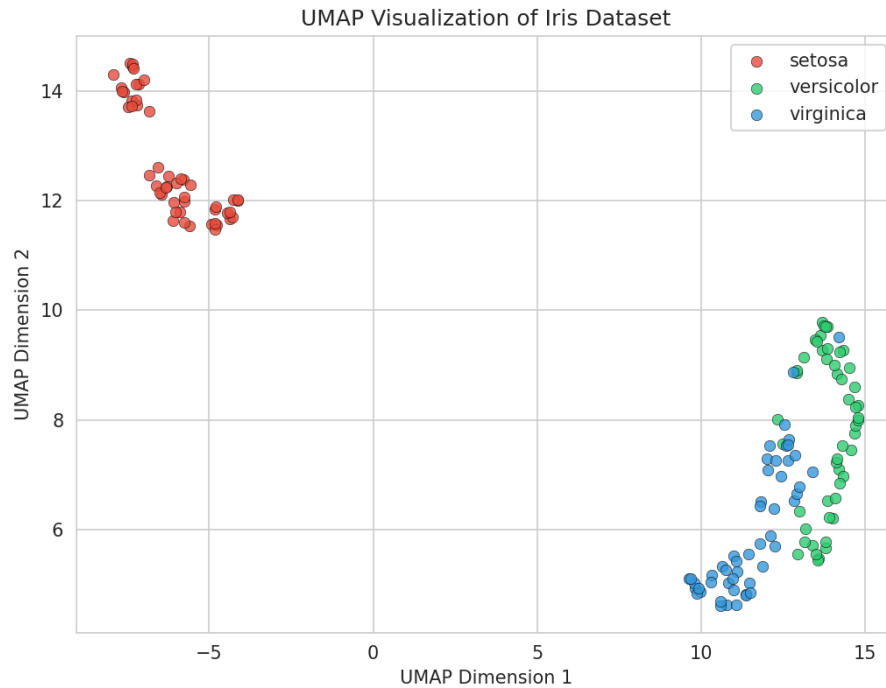
```
import umap
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

iris = load_iris()
X_scaled = StandardScaler().fit_transform(iris.data)

reducer = umap.UMAP(n_components=2, random_state=42)
iris_umap = reducer.fit_transform(X_scaled)

colors = ['#e74c3c', '#2ecc71', '#3498db']
for i, species in enumerate(iris.target_names):
    idx = iris.target == i
    plt.scatter(iris_umap[idx,0], iris_umap[idx,1], label=species, color=colors[i])
plt.title("UMAP Visualization of Iris Dataset")
plt.legend()
plt.show()
```

Output:



Interpretation:

Similar to t-SNE, UMAP separates Setosa completely from the other two species, and forms two adjoining but distinguishable clusters for Versicolor and Virginica. UMAP achieves this separation while running faster than t-SNE and generally preserving more of the global relationships between clusters, which is useful when relative distances between clusters (and not just within-cluster tightness) matter for interpretation.

Question 15: E-Commerce Customer Data — UMAP (60 Attributes)

An e-commerce company has collected customer purchasing behaviour data consisting of 60 attributes. UMAP is used to reduce the dimensions and identify customer clusters.

Python Code:

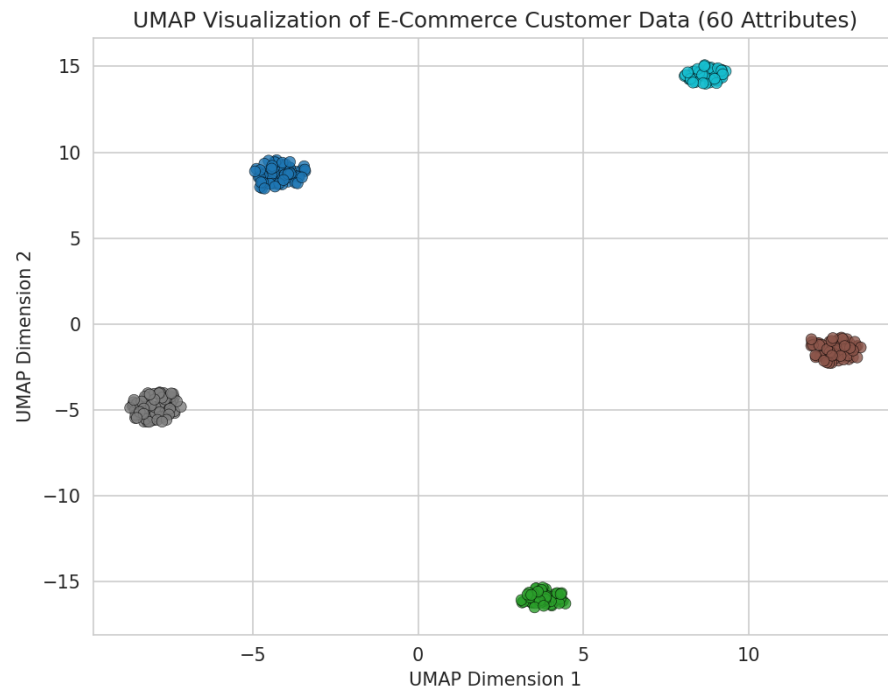
```
import umap
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# ecom_data: customers x 60 purchasing-behaviour attributes
ecom_scaled = StandardScaler().fit_transform(ecom_data)

reducer = umap.UMAP(n_components=2, random_state=42)
ecom_2d = reducer.fit_transform(ecom_scaled)

plt.figure(figsize=(7,5.5))
plt.scatter(ecom_2d[:,0], ecom_2d[:,1], c=ecom_labels, cmap='tab10', edgecolor='k')
plt.title("UMAP Visualization of E-Commerce Customer Data (60 Attributes)")
plt.show()
```

Output:



Interpretation — Customer Clusters Identified:

UMAP reveals five distinct, well-separated customer clusters from the 60-dimensional purchasing-behaviour data. Each cluster likely represents a different customer persona (for example: bargain hunters, brand-loyal repeat buyers, occasional/seasonal shoppers, high-value frequent buyers, and cart-abandoners/browsers). The clear separation confirms that customer purchasing behaviour is not homogeneous and can be reliably segmented, enabling the company to design targeted promotions, personalized recommendations, and retention strategies for each cluster.

Overall Conclusion of the Activity:

Across all 15 tasks, this activity demonstrates the full analytical pipeline for multivariate data: (1) scatterplots and correlation analysis to detect and quantify bivariate relationships, (2) correlograms/heatmaps to spot multicollinearity across many variables at once, and (3) dimensionality-reduction techniques — PCA (linear, variance-preserving), t-SNE (non-linear, local-structure-preserving) and UMAP (non-linear, faster, better global-structure retention) — to visualize and cluster high-dimensional data in two dimensions.